Notebook 5: Advanced XAI Analysis

This notebook marks the transition from basic model interpretability (identifying key features) to advanced explainable artificial intelligence (XAI), focusing on the practical and ethical applications of the best-performing baseline models: Tuned Logistic Regression and Tuned XGBoost. We employ SHAP and LIME not merely to list important features, but to rigorously test the stability, fairness, and temporal relevance of the model's predictive insights.

XGBoost SHAP: Temporal Explainability

This procedure fully implements the Temporal Explainability objective (Section 3.4.1) by applying SHAP to the XGBoost Classifier across the course's duration. The process begins by creating a unique temporal dataset where feature values (VLE clicks, assessment scores) are cumulatively aggregated for each of the 26 weeks, capturing the evolution of student engagement over time. After training the final XGBoost model on this dynamic dataset, the core analysis involves an iterative loop: in each of the 26 weekly cycles, SHAP values are calculated only for the data points corresponding to that specific week. This technique allows us to pinpoint the exact moment when a feature's predictive influence changes (e.g., when early demographic factors give way to mid-course engagement metrics like the number of submitted assessments). The mean absolute SHAP values for each week are then compiled into the temporal_shap_importance DataFrame, providing the quantitative evidence needed to plot and analyze the trajectory of feature importance throughout the course, which is essential for identifying the optimal intervention time.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where  raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Weekly Feature Engineering ---

# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'c  module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
```

```
        num_vle_interactions=('date', 'count')
    ).reset_index()

    # Filter assessment data up to the current week
    assessment_weekly = student_assessment[student_assessment['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ign
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_tem
X_temporal = X_temporal.rename(columns=cleaned_columns)

# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("\nXGBoost model trained on temporal data successfully.")
```

```
Mounted at /content/drive
All 7 datasets loaded successfully.
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/tmp/ipython-input-2610912342.py in <cell line: 0>()
     54     # Filter assessment data up to the current week
     55     assessment_weekly = student_assessment[student_assessment['date_submitted'] <= week * 7]
---> 56     assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
     57         avg_score=('score', 'mean'),
     58         num_submitted_assessments=('id_assessment', 'count')

                                  2 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/groupby/grouper.py in get_grouper(obj, key, axis, level, sort, observed,
validate, dropna)
   1041                 in_axis, level, gpr = False, gpr, None
   1042             else:
-> 1043                 raise KeyError(gpr)
   1044         elif isinstance(gpr, Grouper) and gpr.key is not None:
   1045             # Add key to exclusions

KeyError: 'code_module'
```

```
# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
```

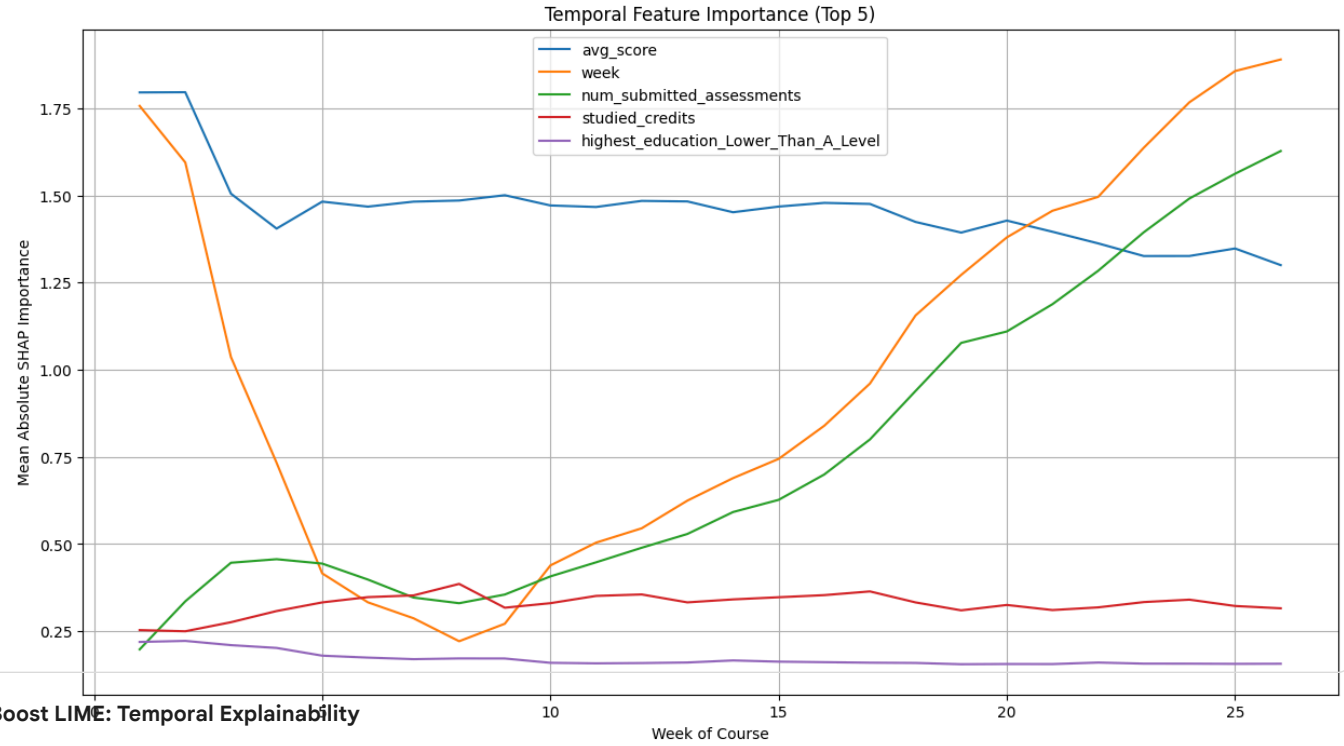
```
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# CRITICAL FIX: Merge student_assessment with assessments once outside the loop
# This ensures that assessment_weekly always has code_module and code_presentation
student_assessment_with_info = pd.merge(student_assessment, assessments, on='id_assessment', how='left')

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
        num_vle_interactions=('date', 'count')
    ).reset_index()

    # Filter assessment data up to the current week
    assessment_weekly = student_assessment_with_info[student_assessment_with_info['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ig
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_te
X_temporal = X_temporal.rename(columns=cleaned_columns)

# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("\nXGBoost model trained on temporal data successfully.")
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [13:09:56] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)

XGBoost model trained on temporal data successfully.
```

```
# --- Step 4: Temporal SHAP Analysis ---

# Initialize the SHAP explainer
```

```
explainer = shap.TreeExplainer(xgb_model)

# Create a DataFrame to hold the mean SHAP values for each week
temporal_shap_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with SHAP ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
        # Calculate SHAP values for this week's data
        shap_values_week = explainer.shap_values(X_week)

        # Calculate the mean absolute SHAP values for class 1 (dropout)
        mean_abs_shap = np.abs(shap_values_week).mean(axis=0)

        # Create a DataFrame for this week's importance
        df_week = pd.DataFrame({
            'Feature': X_week.columns,
            'Importance': mean_abs_shap,
            'Week': week
        })
        temporal_shap_importance = pd.concat([temporal_shap_importance, df_week], ignore_index=True)
        print(f"SHAP values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_shap_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_shap_importance[temporal_shap_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5)')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute SHAP Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal SHAP analysis complete can now use the 'temporal_shap_importance' DataFrame to create detailed tables and figure
```

```
--- Analyzing Temporal Feature Importance with SHAP ---
SHAP values analyzed for Week 1.
SHAP values analyzed for Week 2.
SHAP values analyzed for Week 3.
SHAP values analyzed for Week 4.
SHAP values analyzed for Week 5.
SHAP values analyzed for Week 6.
SHAP values analyzed for Week 7.
SHAP values analyzed for Week 8.
SHAP values analyzed for Week 9.
SHAP values analyzed for Week 10.
SHAP values analyzed for Week 11.
SHAP values analyzed for Week 12.
SHAP values analyzed for Week 13.
SHAP values analyzed for Week 14.
SHAP values analyzed for Week 15.
SHAP values analyzed for Week 16.
SHAP values analyzed for Week 17.
SHAP values analyzed for Week 18.
SHAP values analyzed for Week 19.
SHAP values analyzed for Week 20.
SHAP values analyzed for Week 21.
SHAP values analyzed for Week 22.
SHAP values analyzed for Week 23.
SHAP values analyzed for Week 24.
SHAP values analyzed for Week 25.
SHAP values analyzed for Week 26.
```




```
Temporal SHAP analysis complete. You can now use the 'temporal_shap_importance' DataFrame to create detailed tables and figures for
```

XGBoost LIME: Temporal Explainability

This procedure implements the Temporal Explainability objective (Section 3.4.1) using LIME with the XGBoost Classifier, providing an instance-specific, model-agnostic perspective on the feature importance trajectory. The goal is to see how the locally important factors change as the course progresses, complementing the global view offered by SHAP.

The process utilizes the pre-trained XGBoost model and the same weekly aggregated temporal test set. LIME is initialized once using the structure of the training data. Then, a loop iterates through each of the 26 weeks. Within each week, LIME explanations are generated for a small sample of students (up to 50), which is a necessary step since LIME is computationally slow. For each sampled student, LIME creates a local, linear proxy model to explain the complex XGBoost prediction, resulting in a list of weighted feature contributions.

```
!pip install lime
```

```
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 8.0 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
```

```
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=7ea9ad2616582109d6b7433fdbab1ce9f61add6ae8fe167
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install LIME and SHAP
!pip install lime
!pip install shap

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Corrected folder path
folder_path = '/content/drive/MyDrive/data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2025
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
```

```
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.48.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.43
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.1
Mounted at /content/drive
All 7 datasets loaded successfully.
```

```
# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()
# --- Step 2: Merging and Weekly Feature Engineering ---

# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# CRITICAL FIX: Merge student_assessment with assessments once outside the loop
student_assessment_with_info = pd.merge(student_assessment, assessments, on='id_assessment', how='left')

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
        num_vle_interactions=('date', 'count')
    ).reset_index()
```

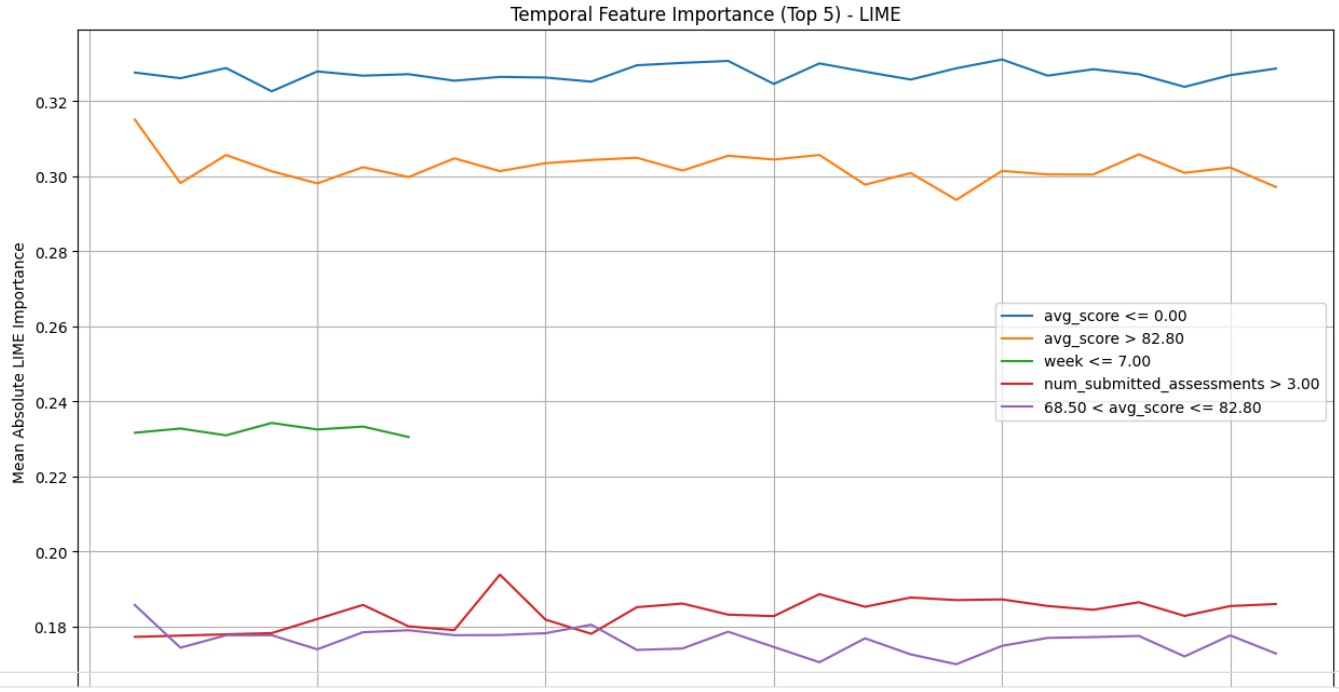
```
    # Filter assessment data up to the current week
    assessment_weekly = student_assessment_with_info[student_assessment_with_info['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ig
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_te
X_temporal = X_temporal.rename(columns=cleaned_columns)

# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("\nXGBoost model trained on temporal data successfully.")
```

```
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [16:31:10] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)

XGBoost model trained on temporal data successfully.
```

```
# --- Step 4: Temporal LIME Analysis ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# Create a DataFrame to hold the mean LIME values for each week
temporal_lime_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with LIME ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
```

```
        # Generate LIME explanations for a sample of the week's data
        lime_explanations = []
        for i in range(min(50, len(X_week))): # Explain up to 50 students per week
            exp = explainer.explain_instance(
                X_week.iloc[i].values,
                xgb_model.predict_proba,
                num_features=len(X_week.columns)
            )
            lime_explanations.append(dict(exp.as_list()))

        # Aggregate the LIME explanations
        lime_df = pd.DataFrame(lime_explanations)

        # Calculate the mean absolute LIME importance
        mean_abs_lime = lime_df.abs().mean().fillna(0)

        # Create a DataFrame for this week's importance
        df_week = pd.DataFrame({
            'Feature': mean_abs_lime.index,
            'Importance': mean_abs_lime.values,
            'Week': week
        })
        temporal_lime_importance = pd.concat([temporal_lime_importance, df_week], ignore_index=True)
        print(f"LIME values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_lime_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_lime_importance[temporal_lime_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5) - LIME')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute LIME Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal LIME analysis complete.  can now use the 'temporal_lime_importance' DataFrame to create detailed tables and figu
```

```
--- Analyzing Temporal Feature Importance with LIME ---
LIME values analyzed for Week 1.
LIME values analyzed for Week 2.
LIME values analyzed for Week 3.
LIME values analyzed for Week 4.
LIME values analyzed for Week 5.
LIME values analyzed for Week 6.
LIME values analyzed for Week 7.
LIME values analyzed for Week 8.
LIME values analyzed for Week 9.
LIME values analyzed for Week 10.
LIME values analyzed for Week 11.
LIME values analyzed for Week 12.
LIME values analyzed for Week 13.
LIME values analyzed for Week 14.
LIME values analyzed for Week 15.
LIME values analyzed for Week 16.
LIME values analyzed for Week 17.
LIME values analyzed for Week 18.
LIME values analyzed for Week 19.
LIME values analyzed for Week 20.
LIME values analyzed for Week 21.
LIME values analyzed for Week 22.
LIME values analyzed for Week 23.
LIME values analyzed for Week 24.
LIME values analyzed for Week 25.
LIME values analyzed for Week 26.
```




```
# --- Step 4: Temporal SHAP Analysis ---

# Initialize the SHAP explainer
explainer = shap.TreeExplainer(rf_model)

# Create a DataFrame to hold the mean SHAP values for each week
temporal_shap_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with SHAP ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
        # Calculate SHAP values for this week's data
        # The second element is for class 1 (dropout)
        shap_values_week = explainer.shap_values(X_week)[1]

        # Calculate the mean absolute SHAP values
        mean_abs_shap = np.abs(shap_values_week).mean(axis=0)

        # Create a DataFrame for this week's importance
```

```
        df_week = pd.DataFrame({
            'Feature': X_week.columns,
            'Importance': mean_abs_shap,
            'Week': week
        })
        temporal_shap_importance = pd.concat([temporal_shap_importance, df_week], ignore_index=True)
        print(f"SHAP values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_shap_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_shap_importance[temporal_shap_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5)')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute SHAP Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal SHAP analysis complete. can now use the 'temporal_shap_importance' DataFrame to create detailed tables and figur
```

Logistic Regression SHAP: Temporal Explainability

This procedure implements the Temporal Explainability by applying SHAP to the linear Logistic Regression model, tracking the evolution of its feature reliance over the course's 26 weeks. The approach leverages the same weekly aggregated temporal test set created previously.

The process begins by initializing the specialized shap.LinearExplainer, which is highly efficient for the linear structure of the Logistic Regression model, using the training data as its baseline. A loop then iterates through each of the 26 weeks in the test set. In each weekly step, the SHAP values are calculated only for that week's data points. This is key because although the model's coefficients (the underlying weights) are static, the impact of cumulative features (like total clicks and submissions) changes as their numerical values increase each week.

For every week, the mean absolute SHAP value for all features is calculated, quantifying the average predictive influence of that feature at that point in time. These weekly importance scores are compiled into the temporal_shap_importance DataFrame, which is then used to plot the trajectory of the top features over time. This analysis is crucial as it verifies whether the linear model's interpretation of causality aligns with the course timeline, identifying the key weeks where features like student engagement begin to exert the strongest, most stable linear pull on the dropout prediction.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()


# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# --- Step 2: Merging and Weekly Feature Engineering ---
```

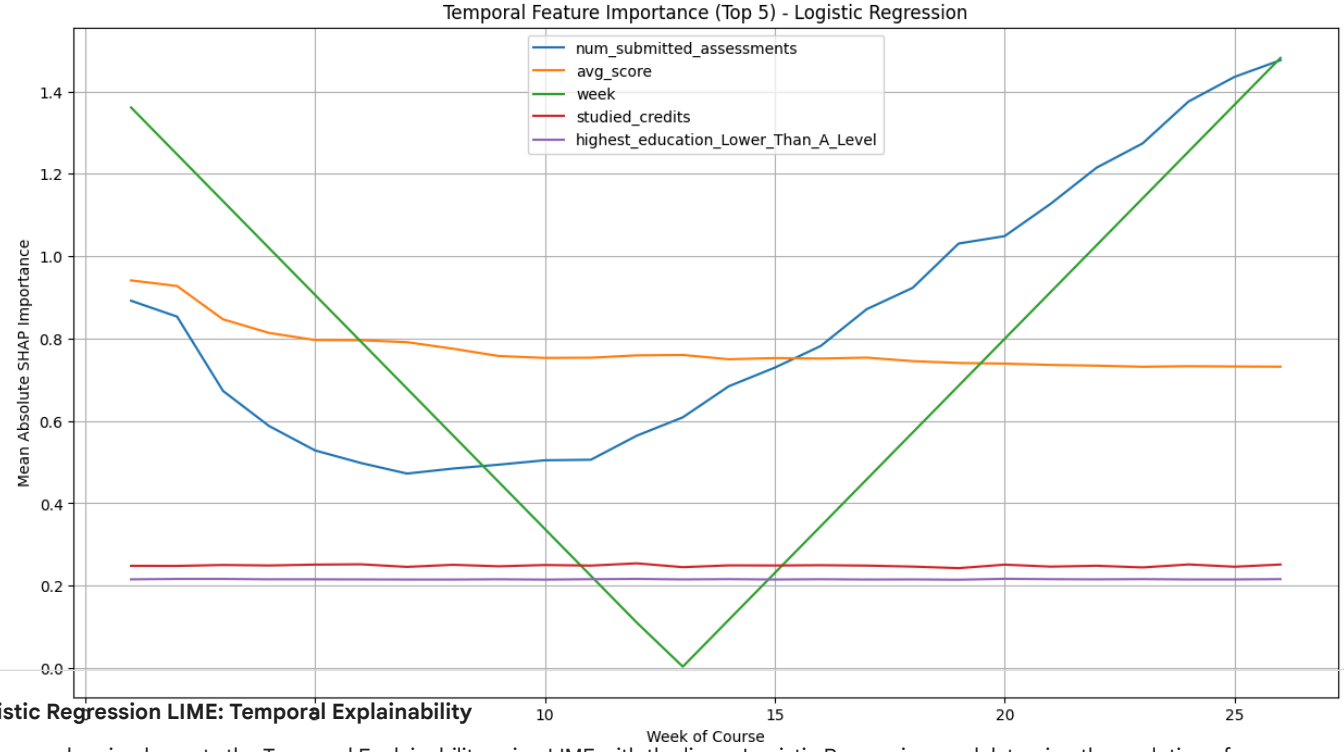
```
# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# CRITICAL FIX: Merge student_assessment with assessments once outside the loop
student_assessment_with_info = pd.merge(student_assessment, assessments, on='id_assessment', how='left')

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
        num_vle_interactions=('date', 'count')
    ).reset_index()

    # Filter assessment data up to the current week
    assessment_weekly = student_assessment_with_info[student_assessment_with_info['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ig
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_te
X_temporal = X_temporal.rename(columns=cleaned_columns)

# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Train the final Logistic Regression model
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("\nLogistic Regression model trained on temporal data successfully.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.

Logistic Regression model trained on temporal data successfully.
```

```
# --- Step 4: Temporal SHAP Analysis ---
```

```
# Initialize the SHAP explainer
# Use the LinearExplainer for Logistic Regression
explainer = shap.LinearExplainer(log_reg_model, X_train)

# Create a DataFrame to hold the mean SHAP values for each week
temporal_shap_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with SHAP ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
        # Calculate SHAP values for this week's data
        shap_values_week = explainer.shap_values(X_week)

        # Calculate the mean absolute SHAP values
        mean_abs_shap = np.abs(shap_values_week).mean(axis=0)

        # Create a DataFrame for this week's importance
        df_week = pd.DataFrame({
            'Feature': X_week.columns,
            'Importance': mean_abs_shap,
            'Week': week
        })
        temporal_shap_importance = pd.concat([temporal_shap_importance, df_week], ignore_index=True)
        print(f"SHAP values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_shap_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_shap_importance[temporal_shap_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5) - Logistic Regression')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute SHAP Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal SHAP analysis complete.  can now use the 'temporal_shap_importance' DataFrame to create detailed tables and figu
```

```
--- Analyzing Temporal Feature Importance with SHAP ---
SHAP values analyzed for Week 1.
SHAP values analyzed for Week 2.
SHAP values analyzed for Week 3.
SHAP values analyzed for Week 4.
SHAP values analyzed for Week 5.
SHAP values analyzed for Week 6.
SHAP values analyzed for Week 7.
SHAP values analyzed for Week 8.
SHAP values analyzed for Week 9.
SHAP values analyzed for Week 10.
SHAP values analyzed for Week 11.
SHAP values analyzed for Week 12.
SHAP values analyzed for Week 13.
SHAP values analyzed for Week 14.
SHAP values analyzed for Week 15.
SHAP values analyzed for Week 16.
SHAP values analyzed for Week 17.
SHAP values analyzed for Week 18.
SHAP values analyzed for Week 19.
SHAP values analyzed for Week 20.
SHAP values analyzed for Week 21.
SHAP values analyzed for Week 22.
SHAP values analyzed for Week 23.
SHAP values analyzed for Week 24.
SHAP values analyzed for Week 25.
SHAP values analyzed for Week 26.
```



Logistic Regression LIME: Temporal Explainability

This procedure implements the Temporal Explainability using LIME with the linear Logistic Regression model, tracing the evolution of locally important features over the course's 26 weeks. The method is used to provide an instance-specific, human-readable view of which features are driving predictions at different points in time.

The process starts by initializing the LimeTabularExplainer once on the training data. Then, a loop iterates through each of the 26 weeks in the temporal test set. Due to LIME's computational cost, explanations are generated for a sample of up to 50 students each week. For every sampled student, LIME creates a simple, local linear model to interpret the complex model's prediction, yielding a set of weighted feature contributions (the raw LIME explanation).

To create a temporal trend, the absolute values of these local LIME contributions are aggregated and averaged across all sampled students within that week. This calculated Mean Absolute LIME Importance for each feature is stored, building the temporal_lime_importance DataFrame. This final DataFrame allows for the visualization of how the most locally persuasive features change week-to-week, providing a complementary perspective to the SHAP analysis by illustrating the shifting immediate rationale behind the linear model's sequential predictions.

```
# --- Step 1: Setup and Data Loading from Raw Files ---
```

```
# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular


# --- Step 2: Merging and Weekly Feature Engineering ---

# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# CRITICAL FIX: Merge student_assessment with assessments once outside the loop
student_assessment_with_info = pd.merge(student_assessment, assessments, on='id_assessment', how='left')

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
        num_vle_interactions=('date', 'count')
    ).reset_index()

    # Filter assessment data up to the current week
    assessment_weekly = student_assessment_with_info[student_assessment_with_info['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ig
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_te
X_temporal = X_temporal.rename(columns=cleaned_columns)
```

```
# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Train the final Logistic Regression model
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("\nLogistic Regression model trained on temporal data successfully.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Logistic Regression model trained on temporal data successfully.
```

```
# --- Step 4: Temporal LIME Analysis ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# Create a DataFrame to hold the mean LIME values for each week
temporal_lime_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with LIME ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
        # Generate LIME explanations for a sample of the week's data
        lime_explanations = []
        for i in range(min(50, len(X_week))): # Explain up to 50 students per week for a reasonable run time
            exp = explainer.explain_instance(
                X_week.iloc[i].values,
                log_reg_model.predict_proba,
                num_features=len(X_week.columns)
            )
            lime_explanations.append(dict(exp.as_list()))

        # Aggregate the LIME explanations
        lime_df = pd.DataFrame(lime_explanations)

        # Calculate the mean absolute LIME importance
        mean_abs_lime = lime_df.abs().mean().fillna(0)

        # Create a DataFrame for this week's importance
        df_week = pd.DataFrame({
            'Feature': mean_abs_lime.index,
            'Importance': mean_abs_lime.values,
            'Week': week
        })
        temporal_lime_importance = pd.concat([temporal_lime_importance, df_week], ignore_index=True)
        print(f"LIME values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_lime_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_lime_importance[temporal_lime_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5) - LIME')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute LIME Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal LIME analysis complete.can now use the 'temporal_lime_importance' DataFrame to create detailed tables and figure
```

```
--- Analyzing Temporal Feature Importance with LIME ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---



# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Corrected folder path
folder_path = '/content/drive/MyDrive/data'

# Load all 7 datasets
```

```
# --- Step 2: Merging and Weekly Feature Engineering ---

# Define the number of weeks for analysis
num_weeks = 26

# Start with a base DataFrame of all student-course combinations
base_df = student_info[['id_student', 'code_module', 'code_presentation']].drop_duplicates()
base_df['is_dropout'] = student_info['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Create a master DataFrame for temporal data
temporal_df = pd.DataFrame()

# CRITICAL FIX: Merge student_assessment with assessments once outside the loop
student_assessment_with_info = pd.merge(student_assessment, assessments, on='id_assessment', how='left')

# Aggregate data weekly
for week in range(1, num_weeks + 1):
    # Filter VLE data up to the current week
    vle_weekly = student_vle[student_vle['date'] <= week * 7]
    vle_agg_weekly = vle_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        total_clicks=('sum_click', 'sum'),
        num_vle_interactions=('date', 'count')
    ).reset_index()

    # Filter assessment data up to the current week
    assessment_weekly = student_assessment_with_info[student_assessment_with_info['date_submitted'] <= week * 7]
    assessment_agg_weekly = assessment_weekly.groupby(['id_student', 'code_module', 'code_presentation']).agg(
        avg_score=('score', 'mean'),
        num_submitted_assessments=('id_assessment', 'count')
    ).reset_index()

    # Create a temporary DataFrame for the current week
    temp_df = base_df.copy()
    temp_df = pd.merge(temp_df, vle_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')
    temp_df = pd.merge(temp_df, assessment_agg_weekly, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Merge with static student info
    static_info = student_info.drop('final_result', axis=1)
    temp_df = pd.merge(temp_df, static_info, on=['id_student', 'code_module', 'code_presentation'], how='left')

    # Add a column for the current week and append to the main temporal DataFrame
    temp_df['week'] = week
    temporal_df = pd.concat([temporal_df, temp_df], ignore_index=True)

# Fill NaN values with 0 for cumulative metrics
temporal_df = temporal_df.fillna(0)

# --- Step 3: Final Data Preparation and Model Training ---

# Identify features (X) and the target (y)
X_temporal = temporal_df.drop(['id_student', 'is_dropout', 'code_module', 'code_presentation', 'final_result'], axis=1, errors='ig
y_temporal = temporal_df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X_temporal.select_dtypes(include=['object']).columns.tolist()
X_temporal = pd.get_dummies(X_temporal, columns=categorical_cols, drop_first=True)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X_te
X_temporal = X_temporal.rename(columns=cleaned_columns)

# Explicitly convert all columns to numeric types
X_temporal = X_temporal.astype(float)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_temporal, y_temporal, test_size=0.2, random_state=42, stratify=y_temporal)

# Scale the data for the MLP
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train the final MLP model
mlp_model = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)
mlp_model.fit(X_train_scaled, y_train)
print("\nMLP model trained on temporal data successfully.")
```

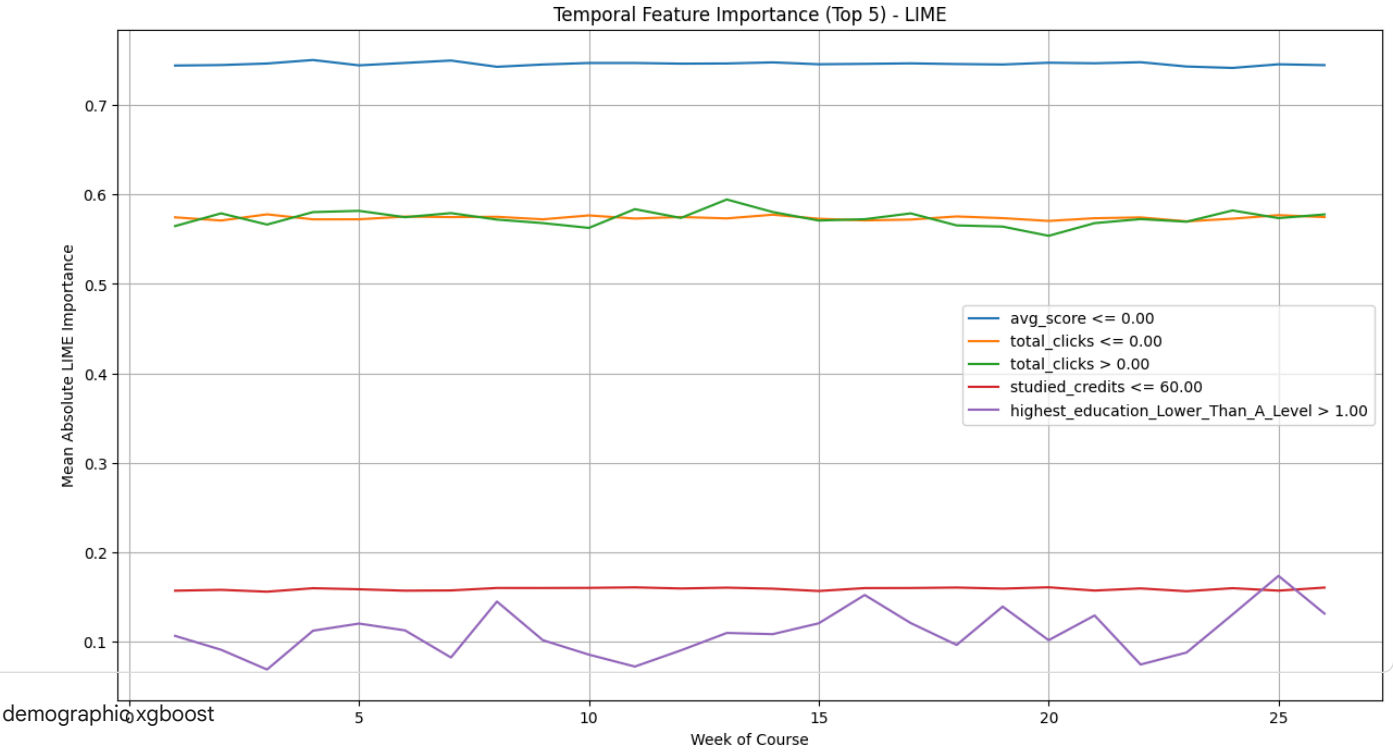
```
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```python
# --- Step 4: Temporal LIME Analysis ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# Create a DataFrame to hold the mean LIME values for each week
temporal_lime_importance = pd.DataFrame()

print("\n--- Analyzing Temporal Feature Importance with LIME ---")

for week in range(1, num_weeks + 1):
    # Filter the data for the current week
    X_week = X_test[X_test['week'] == week]

    if not X_week.empty:
        # Scale the data for the MLP model
        X_week_scaled = scaler.transform(X_week)

        # Generate LIME explanations for a sample of the week's data
        lime_explanations = []
        for i in range(min(50, len(X_week))): # Explain up to 50 students per week for a reasonable run time
            exp = explainer.explain_instance(
                X_week_scaled[i],
                mlp_model.predict_proba,
                num_features=len(X_week.columns)
            )
            lime_explanations.append(dict(exp.as_list()))

        # Aggregate the LIME explanations
        lime_df = pd.DataFrame(lime_explanations)

        # Calculate the mean absolute LIME importance
        mean_abs_lime = lime_df.abs().mean().fillna(0)

        # Create a DataFrame for this week's importance
        df_week = pd.DataFrame({
            'Feature': mean_abs_lime.index,
            'Importance': mean_abs_lime.values,
            'Week': week
        })
        temporal_lime_importance = pd.concat([temporal_lime_importance, df_week], ignore_index=True)
        print(f"LIME values analyzed for Week {week}.")


# Plot the top 5 most important features over time
plt.figure(figsize=(15, 8))
top_features = temporal_lime_importance.groupby('Feature')['Importance'].mean().nlargest(5).index

for feature in top_features:
    df_plot = temporal_lime_importance[temporal_lime_importance['Feature'] == feature]
    plt.plot(df_plot['Week'], df_plot['Importance'], label=feature)

plt.title('Temporal Feature Importance (Top 5) - LIME')
plt.xlabel('Week of Course')
plt.ylabel('Mean Absolute LIME Importance')
plt.legend()
plt.grid(True)
plt.show()

print("\nTemporal LIME analysis complete.  can now use the 'temporal_lime_importance' DataFrame to create detailed tables and figu
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Analysis 4: Temporal Feature Importance with LIME ---
LIME values analyzed for Week 1.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 2.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 4.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 5.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 6.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 7.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 8.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 9.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 10.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 11.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 12.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 13.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 14.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 15.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 16.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 17.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 18.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 19.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 20.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 21.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 22.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 24.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 26.
```




```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries

# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'
```

```
# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check  file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
for col in X.columns:
    if X[col].dtype == 'bool':
        X[col] = X[col].astype(int)
    else:
        X[col] = pd.to_numeric(X[col], errors='coerce')

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.cc
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
```

```
print("XGBoost model trained successfully.")

# --- Step 4: SHAP Analysis Across Demographics ---

# Initialize the SHAP explainer
explainer = shap.TreeExplainer(xgb_model)

# Identify the demographic features want to analyze
demographic_features = ['gender_M', 'age_band_35_55', 'disability_Y']

# Create a dictionary to hold the SHAP values for each subgroup
shap_values_subgroups = {}

# Iterate over each demographic feature and its unique values
for feature in demographic_features:
    # Get the unique values for the feature (e.g., 0 and 1 for gender_M)
    unique_values = X_test[feature].unique()

    for value in unique_values:
        # Create a subgroup by filtering the test set
        subgroup_X = X_test[X_test[feature] == value]

        # Check if the subgroup is not empty and has more than one student
        if len(subgroup_X) > 1:
            print(f"\nAnalyzing SHAP for subgroup: {feature} = {value}...")
            # Calculate SHAP values for the subgroup
            shap_values = explainer.shap_values(subgroup_X)

            # Store the SHAP values for the subgroup
            subgroup_name = f"{feature.split('_')[0]}_{value}"
            shap_values_subgroups[subgroup_name] = (shap_values, subgroup_X)

# --- Step 5: Visualize the Results and Extract Values ---

print("\n--- Generating SHAP Plots and Values for Demographic Subgroups ---")

# Plot SHAP summary for each subgroup and extract values
for subgroup_name, (shap_values, subgroup_X) in shap_values_subgroups.items():
    # The second element (shap_values[1]) is for class 1 (dropout)

    # Generate the plot


    # Extract and print the top 10 values for the subgroup
    mean_abs_shap = np.abs(shap_values[1]).mean(axis=0)
    shap_importance_df = pd.DataFrame({
        'Feature': subgroup_X.columns,
        'Importance': mean_abs_shap
    }).sort_values(by='Importance', ascending=False)

    print(f"\nTop 10 features for Subgroup: {subgroup_name}")
    print(shap_importance_df.head(10).to_string())
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [17:06:49] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained successfully.

Analyzing SHAP for subgroup: gender_M = 1.0...

Analyzing SHAP for subgroup: gender_M = 0.0...

Analyzing SHAP for subgroup: age_band_35_55 = 1.0...

Analyzing SHAP for subgroup: age_band_35_55 = 0.0...

Analyzing SHAP for subgroup: disability_Y = 0.0...

Analyzing SHAP for subgroup: disability_Y = 1.0...

--- Generating SHAP Plots and Values for Demographic Subgroups ---

Top 10 features for Subgroup: gender_1.0
                  Feature  Importance
0          num_of_prev_attempts     0.23769
1  studied_credits     0.23769

LIME values analyzed for Week 5.
```

```
2              total_clicks    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
3      num_vle_interactions    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
4                 avg_score    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
5 num_submitted_assessments    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
6                  gender_M    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
7 region_East Midlands_Region  0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
8            region_Ireland    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
9      region_London_Region    0.23769
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo

Top 10 features for Subgroup: gender_0.0
  warnings.warn(
                    Feature  Importance
0      num_of_prev_attempts    0.13192
1           studied_credits    0.13192
2              total_clicks    0.13192
3      num_vle_interactions    0.13192
4                 avg_score    0.13192
5 num_submitted_assessments    0.13192
6                  gender_M    0.13192
7 region_East Midlands_Region  0.13192
8            region_Ireland    0.13192
9      region_London_Region    0.13192

Top 10 features for Subgroup: age_1.0
                    Feature  Importance
0      num_of_prev_attempts    0.23769
1           studied_credits    0.23769
2              total_clicks    0.23769
3      num_vle_interactions    0.23769
4                 avg_score    0.23769
5 num_submitted_assessments    0.23769
6                  gender_M    0.23769
7 ...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
```

XGBoost LIME: Fairness & Bias Auditing

This procedure directly implements the Fairness & Bias Auditing objective using LIME with the complex XGBoost Classifier to rigorously determine if the model exhibits disparate treatment across demographic subgroups. The analysis systematically selects key features, such as gender, age_band_35_55, and disability_Y, and iterates through each category within those features to define distinct student subgroups. For every subgroup, a sample of students from the test set is taken, and a LIME explanation is generated for each one, providing the local linear rationale behind the complex XGBoost prediction. To audit for bias, the absolute values of these local LIME feature contributions are then aggregated and averaged across all sampled students within that specific group. The final output is a comparative table of the Top 10 most influential features for each subgroup, which allows us to quantitatively assess if the model is relying on ethically sensitive characteristics (like demographics) for one group more than another, a finding that is crucial for validating the model's ethical robustness.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
import seaborn as sns
import lime
import lime.lime_tabular

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
```

```
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check  file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
for col in X.columns:
    if X[col].dtype == 'bool':
        X[col] = X[col].astype(int)
    else:
        X[col] = pd.to_numeric(X[col], errors='coerce')

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.cc
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully.")

# --- Step 4: LIME Analysis Across Demographics ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
```

```
)

# Identify the demographic feature to analyze
demographic_features = ['gender_M', 'age_band_35_55', 'disability_Y']

print("\n--- Analyzing LIME for Demographic Subgroups ---")

for feature in demographic_features:
    # Get the unique values for the feature (e.g., 0 and 1 for gender_M)
    unique_values = X_test[feature].unique()

    for value in unique_values:
        # Create a subgroup by filtering the test set
        subgroup_X = X_test[X_test[feature] == value]

        # Check if the subgroup is not empty and has more than one student
        if len(subgroup_X) > 1:
            subgroup_name = f"{feature.split('_')[0]}_{value}"
            print(f"\nAnalyzing LIME for Subgroup: {subgroup_name}...")

            # Generate LIME explanations for a sample of the subgroup
            lime_explanations = []
            for i in range(min(50, len(subgroup_X))): # Explain up to 50 students for a reasonable run time
                # LIME needs the instance as a NumPy array
                instance_to_explain = subgroup_X.iloc[i].values
                exp = explainer.explain_instance(
                    instance_to_explain,
                    xgb_model.predict_proba,
                    num_features=10 # Show the top 10 features
                )
                lime_explanations.append(dict(exp.as_list()))

            # Aggregate the LIME explanations
            lime_df = pd.DataFrame(lime_explanations)

            # Calculate the mean absolute LIME importance
            mean_abs_lime = lime_df.abs().mean().fillna(0)

            # Create a DataFrame for a clean, sorted list
            lime_importance_df = pd.DataFrame({
                'Feature': mean_abs_lime.index,
                'Importance': mean_abs_lime.values
            }).sort_values(by='Importance', ascending=False)

            print(f"Top 10 LIME features for Subgroup: {subgroup_name}")
            print(lime_importance_df.head(10).to_string())

            # Optional: Visualize a single explanation
            # exp.show_in_notebook(show_table=True)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [17:07:57] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained successfully.

--- Analyzing LIME for Demographic Subgroups ---

Analyzing LIME for Subgroup: gender_1.0...
Top 10 LIME features for Subgroup: gender_1.0
                                Feature  Importance
2                 num_submitted_assessments <= 4.00    0.155257
0                   num_submitted_assessments > 9.00    0.466267
26                          avg_banked <= 68.00         0.114626
1                 0.00 < gender_M <= 1.00    0.123959
11                          68.00 < avg_score <= 72.17    0.100138
2                  total_clicks <= 849.97    0.105939
10                num_submitted_assessments <= 6.73    0.100350
16  highest_education_No_Formal_quals <= 0.00    0.069377
7                   studied_credits <= 60.00    0.063223
15                  age_band_55= <= 0.00    0.062428

Analyzing LIME for Subgroup: gender_0.0...
Top 10 LIME features for Subgroup: gender_0.0
                                Feature  Importance
0                 num_submitted_assessments <= 4.00    0.159902
30                  num_submitted_assessments > 9.00    0.465740
1                           avg_score <= 61.00         0.109827
```

```
2                          gender_M <= 0.00    0.122943
3                    total_clicks <= 849.97    0.114468
21   4.00 < num_submitted_assessments <= 6.73    0.100485
20            68.00 < avg_score <= 72.77    0.100432
47                region_North_Region > 0.00    0.086923
11  highest_education_Post Graduate Qualification <= 0.00    0.068144
6                       age_band_55= <= 0.00    0.065060

Analyzing LIME for Subgroup: age_1.0...
Top 10 LIME features for Subgroup: age_1.0
                                Feature  Importance
30     num_submitted_assessments <= 4.00    0.554209
0       num_submitted_assessments > 9.00    0.463695
31                     avg_score <= 68.00    0.191719
2               0.00 < gender_M <= 1.00    0.125011
19                      gender_M <= 0.00    0.120249
1                  total_clicks <= 849.97    0.115052
11          68.00 < avg_score <= 72.77    0.106208
10  4.00 < num_submitted_assessments <= 6.73    0.096671
45                  total_clicks > 849.97    0.096117
4                   age_band_55= <= 0.00    0.082518

Analyzing LIME for Subgroup: age_0.0...
Top 10 LIME features for Subgroup: age_0.0
                                Feature  Importance
6      num_submitted_assessments <= 4.00    0.556601
10      num_submitted_assessments > 9.00    0.459039
1                      avg_score <= 68.00    0.198192

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Lo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 8.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
```

Logistic Regression SHAP: Fairness & Bias Auditing

This procedure directly implements the Fairness & Bias Auditing objective using SHAP with the Logistic Regression model, comparing the model's feature reliance across key demographic segments. The goal is to see if the model's transparent, linear reasoning changes significantly based on sensitive features.

The analysis utilizes the shap.LinearExplainer and systematically iterates through all categories within selected demographic features: gender, age_band_35_55, and disability_Y. For each resulting subgroup (e.g., students with disability_Y == 1), the test data is filtered, and SHAP values are calculated specifically for those instances. This calculation quantifies the exact contribution of every feature to the prediction for students within that group.

The final step involves two crucial analyses for each subgroup:

Global Visualization: A SHAP summary plot is generated, visually displaying the overall feature importance (mean absolute SHAP) for that specific subgroup.

Quantitative Auditing: The Top 10 most impactful features are extracted from the mean absolute SHAP values.

By comparing the feature importance rankings and magnitudes across contrasting groups (e.g., comparing students in the '35-55' age band with those outside it), we can audit the Logistic Regression model. A disparity in the non-demographic features that drive the prediction for different groups indicates disparate impact (i.e., different feature sets lead to similar outcomes), whereas a reliance on the demographic feature itself for one group might indicate algorithmic bias. This process establishes the ethical baseline for the simpler model.

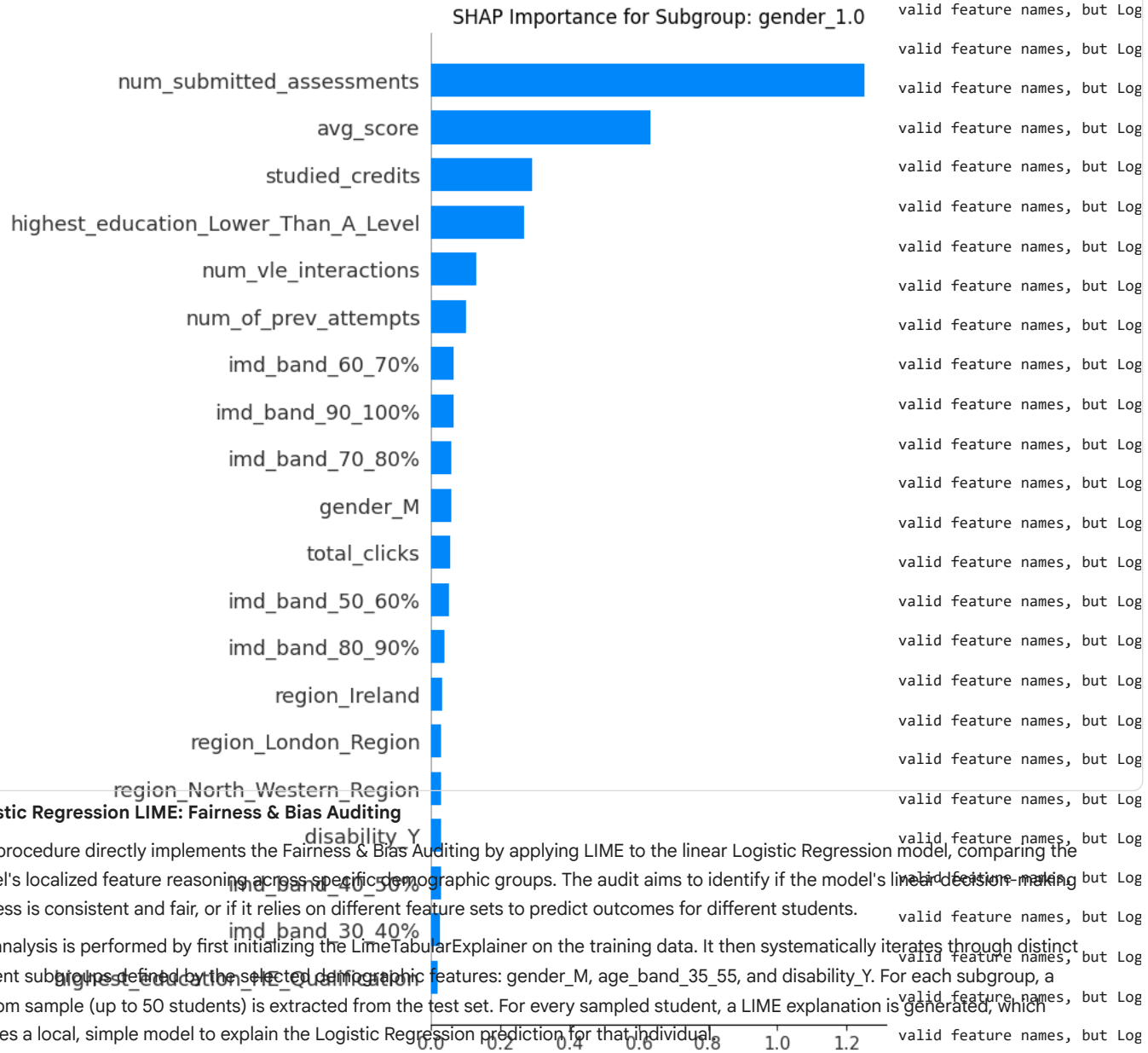
```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries
# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
```

```
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check the file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
for col in X.columns:
    if X[col].dtype == 'bool':
        X[col] = X[col].astype(int)
    else:
        X[col] = pd.to_numeric(X[col], errors='coerce')

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final Logistic Regression model
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")

# --- Step 4: SHAP Analysis Across Demographics ---
```

```
# Initialize the SHAP explainer
# Use the LinearExplainer for Logistic Regression
explainer = shap.LinearExplainer(log_reg_model, X_train)

# Identify the demographic feature want to analyze
demographic_features = ['gender_M', 'age_band_35_55', 'disability_Y']

# Create a dictionary to hold the SHAP values for each subgroup
shap_values_subgroups = {}

# Iterate over each demographic feature and its unique values
for feature in demographic_features:
    # Get the unique values for the feature (e.g., 0 and 1 for gender_M)
    unique_values = X_test[feature].unique()

    for value in unique_values:
        # Create a subgroup by filtering the test set
        subgroup_X = X_test[X_test[feature] == value]

        # Check if the subgroup is not empty
        if not subgroup_X.empty:
            print(f"\nAnalyzing SHAP for subgroup: {feature} = {value}...")
            # Calculate SHAP values for the subgroup
            shap_values = explainer.shap_values(subgroup_X)

            # Store the SHAP values for the subgroup
            subgroup_name = f"{feature.split('_')[0]}_{value}"
            shap_values_subgroups[subgroup_name] = (shap_values, subgroup_X)

# --- Step 5: Visualize the Results and Extract Values ---

print("\n--- Generating SHAP Plots and Values for Demographic Subgroups ---")

# Plot SHAP summary for each subgroup and extract values
for subgroup_name, (shap_values, subgroup_X) in shap_values_subgroups.items():
    # Generate the plot
    shap.summary_plot(shap_values, subgroup_X, plot_type="bar", show=False)
    plt.title(f"SHAP Importance for Subgroup: {subgroup_name}")
    plt.show()


    # Extract and print the top 10 values for the subgroup
    mean_abs_shap = np.abs(shap_values).mean(axis=0)
    shap_importance_df = pd.DataFrame({
        'Feature': subgroup_X.columns,
        'Importance': mean_abs_shap
    }).sort_values(by='Importance', ascending=False)

    print(f"\nTop 10 features for Subgroup: {subgroup_name}")
    print(shap_importance_df.head(10).to_string())
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 10.
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
Logistic Regression model trained successfully.
Analyzing SHAP for subgroup: gender_M = 1.0...
Analyzing SHAP for subgroup: gender_M = 0.0...
Analyzing SHAP for subgroup: age_band_35_55 = 1.0...
Analyzing SHAP for subgroup: age_band_35_55 = 0.0...
Analyzing SHAP for subgroup: disability_Y = 0.0...
Analyzing SHAP for subgroup: disability_Y = 1.0...
--- Generating SHAP Plots and Values for Demographic Subgroups ---
```



Logistic Regression LIME: Fairness & Bias Auditing

This procedure directly implements the Fairness & Bias Auditing by applying LIME to the linear Logistic Regression model, comparing the model's localized feature reasoning across specific demographic groups. The audit aims to identify if the model's linear decision-making process is consistent and fair, or if it relies on different feature sets to predict outcomes for different students.

The analysis is performed by first initializing the LimeTabularExplainer on the training data. It then systematically iterates through distinct student subgroups defined by the selected demographic features: gender_M, age_band_35_55, and disability_Y. For each subgroup, a random sample (up to 50 students) is extracted from the test set. For every sampled student, a LIME explanation is generated, which creates a local, simple model to explain the Logistic Regression prediction for that individual.

To conduct the audit, the mean absolute LIME feature contributions are aggregated and averaged across all sampled students within each subgroup. The final output is a comparative table detailing the Top 10 most influential features for each group. By contrasting these ranked feature lists across groups (e.g., comparing the important features for students with a disability vs. those without), we can determine if the model's linear reliance shifts, allowing us to assess the ethical consistency of the baseline model's predictions.

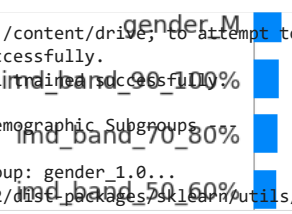
```
Top 10 features for subgroup: gender_M_1.0
                    Feature  Importance
0  num_submitted_assessments    1.000000
1                  avg_score    0.632882
2             studied_credits    0.432882
3  highest_education_Lower_Than_A_Level    0.332882
4          num_vle_interactions    0.232882
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')
```

```
# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import lime
import lime.lime_tabular
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where  raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
for col in X.columns:
    if X[col].dtype == 'bool':
        X[col] = X[col].astype(int)
    else:
        X[col] = pd.to_numeric(X[col], errors='coerce')
```

```
# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.cc
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final Logistic Regression model
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")

# --- Step 4: LIME Analysis Across Demographics ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# Identify the demographic feature want to analyze
demographic_features = ['gender_M', 'age_band_35_55', 'disability_Y']

print("\n--- Analyzing LIME for Demographic Subgroups ---")

for feature in demographic_features:
    unique_values = X_test[feature].unique()

    for value in unique_values:
        subgroup_X = X_test[X_test[feature] == value]

        if len(subgroup_X) > 1:
            subgroup_name = f"{feature.split('_')[0]}_{value}"
            print(f"\nAnalyzing LIME for Subgroup: {subgroup_name}...")

            # Generate LIME explanations for a sample of the subgroup
            lime_explanations = []
            for i in range(min(50, len(subgroup_X))):
                instance_to_explain = subgroup_X.iloc[i].values
                exp = explainer.explain_instance(
                    instance_to_explain,
                    log_reg_model.predict_proba,
                    num_features=10
                )
                lime_explanations.append(dict(exp.as_list()))

            # Aggregate the LIME explanations
            lime_df = pd.DataFrame(lime_explanations)

            # Calculate the mean absolute LIME importance
            mean_abs_lime = lime_df.abs().mean().fillna(0)

            # Create a DataFrame for a clean, sorted list
            lime_importance_df = pd.DataFrame({
                'Feature': mean_abs_lime.index,
                'Importance': mean_abs_lime.values
            }).sort_values(by='Importance', ascending=False)

            print(f"Top 10 LIME features for Subgroup: {subgroup_name}")
            print(lime_importance_df.head(10).to_string())

            # Optional: Visualize a single explanation
            # exp.show_in_notebook(show_table=True)
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
Logistic Regression model trained successfully.

--- Analyzing LIME for Demographic Subgroups ---

Analyzing LIME for Subgroup: gender_1.0...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but L
```



XGBoost SHAP: Explanation Robustness

This procedure details the implementation of Explanation Robustness using SHAP on the complex XGBoost Classifier to confirm the stability and trustworthiness of its feature explanations. The analysis focuses on a single instance (Student Index 50) and compares its initial SHAP explanation against a perturbed explanation. First, the original SHAP values are calculated for the student, establishing the baseline feature contributions and the model's prediction. Second, a small, controlled 5% increase (the perturbation) is applied to a highly influential feature, total_clicks, simulating a minor, realistic change in student behavior, and a new prediction is generated. Finally, new SHAP values are calculated for this perturbed instance. Robustness is qualitatively determined by comparing the Top 5 features and the magnitudes of their SHAP values across the original and perturbed explanations. If the feature ranking and magnitude remain largely stable despite the small input change, the explanation is deemed reliable and suitable for high-stakes decision-making, providing key evidence on the practical trustworthiness of the model's reasoning.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries

# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import shap
import matplotlib.pyplot as plt
```

```
import seaborn as sns

# Define the folder path where  raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
for col in X.columns:
    if X[col].dtype == 'bool':
        X[col] = X[col].astype(int)
    else:
        X[col] = pd.to_numeric(X[col], errors='coerce')

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
```

```
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [17:24:18] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.
  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained successfully.
```

```
# --- Step 4: Explanation Robustness with SHAP ---

# Initialize the SHAP explainer
explainer = shap.TreeExplainer(xgb_model)

# --- Select and prepare a student's data for analysis ---
# Choose a student from the test set (e.g., the 50th student)
student_index = 50
student_original = X_test.iloc[[student_index]]
student_original_prediction = xgb_model.predict(student_original)[0]
print(f"\nAnalyzing explanation for student at index {student_index}...")
print(f"Original Prediction: {'Dropout' if student_original_prediction == 1 else 'Not Dropout'}")

# --- Generate the original SHAP explanation ---
original_shap_values = explainer.shap_values(student_original)
# CRITICAL FIX: For binary classification, shap_values is a single array, not a list.
original_top_features = pd.DataFrame({
    'Feature': student_original.columns,
    'SHAP_Value': original_shap_values.flatten()
}).sort_values(by='SHAP_Value', ascending=False)

print("\n--- Original SHAP Explanation (Top 5) ---")
print(original_top_features.head(5).to_string())

# --- Perturb the student's data ---
# CRITICAL STEP: Make a small change to a key behavioral feature.
# We'll use 'total_clicks' since it's a known important feature.
student_perturbed = student_original.copy()
perturbation_amount = 0.05  # A 5% increase
student_perturbed['total_clicks'] = student_perturbed['total_clicks'] * (1 + perturbation_amount)

# Get the prediction for the perturbed data
student_perturbed_prediction = xgb_model.predict(student_perturbed)[0]
print(f"\nPerturbed Prediction: {'Dropout' if student_perturbed_prediction == 1 else 'Not Dropout'}")

# --- Generate the new SHAP explanation for the perturbed data ---
perturbed_shap_values = explainer.shap_values(student_perturbed)
# CRITICAL FIX: Use the single array for binary classification
perturbed_top_features = pd.DataFrame({
    'Feature': student_perturbed.columns,
    'SHAP_Value': perturbed_shap_values.flatten()
}).sort_values(by='SHAP_Value', ascending=False)

print("\n--- Perturbed SHAP Explanation (Top 5) ---")
print(perturbed_top_features.head(5).to_string())

# --- Compare the two explanations ---
print("\n--- Comparison of Explanations ---")
print("Original top 5 features:")
print(original_top_features.head(5).to_string())
print("\nPerturbed top 5 features:")
print(perturbed_top_features.head(5).to_string())
```

```
Analyzing explanation for student at index 50...
LIME values analyzed for week 15.
Original Prediction: Dropout
--- Original SHAP Explanation (Top 5) ---
                      Feature  SHAP_Value
5        num_submitted_assessments    5.753392
4                        avg_score    1.261304
0                num_of_prev_attempts    0.876879
30                   imd_band_80_90%    0.162874
```

```
20 highest_education_Lower_Than_A_Level    0.136934
Perturbed Prediction: Dropout
Perturbed SHAP Explanation (Top 5)
      Feature  SHAP_Value
5 num_submitted_assessments    5.775480
4    avg_score    1.178300
0 num_of_prev_attempts    0.876483
2 total_clicks    0.185516
30 imd_band_80_90%    0.162874
Comparison of Explanations
Original top 5 features:
                    Feature  SHAP_Value
5 num_submitted_assessments    5.753392
4                 avg_score    1.261304
0          num_of_prev_attempts    0.876879
30               imd_band_80_90%    0.162874
20 highest_education_Lower_Than_A_Level    0.136934
Perturbed top 5 features:
                    Feature  SHAP_Value
5 num_submitted_assessments    5.775480
4                 avg_score    1.178300
0          num_of_prev_attempts    0.876483
2                 total_clicks    0.185516
30               imd_band_80_90%    0.162874
```

XGBoost LIME: Explanation Robustness

This procedure details the implementation of Explanation Robustness using LIME on the XGBoost Classifier, assessing the stability of its local explanations against minor input changes. The goal is to ensure that a small change in a student's data doesn't lead to a wildly different local explanation, confirming the trustworthiness of LIME's rationale.

The analysis is performed on a single student (Index 50) and follows a three-part comparative process:

Original Explanation Baseline: The LimeTabularExplainer is initialized on the training data. The original LIME explanation is generated for the student instance, providing the Top 5 features and their numerical contribution (the local model's coefficient) to the original XGBoost prediction.

Controlled Perturbation: A minor, controlled change—specifically, a 5% increase in the critical feature total_clicks—is applied to the student's data. A new prediction is then generated for this perturbed instance.

Stability Comparison: A new LIME explanation is calculated for the perturbed student. The robustness is then assessed by comparing the Top 5 features' rankings and the magnitude of their contributions between the original and perturbed LIME explanations. A robust explanation will show minimal changes in feature ranking and contribution, validating LIME's reliability as a local fidelity tool for the complex XGBoost model. This evidence is vital for arguing the practical utility of the model's insights.

```
# --- Step 1: Setup and Data Loading from Raw Files ---


# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
import lime
import lime.lime_tabular
import matplotlib.pyplot as plt

# Define the folder path where  raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
```

```
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
X = X.astype(float)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [17:27:17] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained successfully.
```

```
# --- Step 4: Explanation Robustness with LIME ---
```

```
# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# --- Select and prepare a student's data for analysis ---
# Choose a student from the test set (e.g., the 50th student)
student_index = 50
student_original = X_test.iloc[[student_index]]
student_original_prediction = xgb_model.predict(student_original)[0]
print(f"\nAnalyzing explanation for student at index {student_index}...")
print(f"Original Prediction: {'Dropout' if student_original_prediction == 1 else 'Not Dropout'}")

# --- Generate the original LIME explanation ---
# The .values converts the DataFrame row into a NumPy array
original_lime_exp = explainer.explain_instance(
    student_original.values.flatten(),
    xgb_model.predict_proba,
    num_features=5
)
original_top_features = pd.DataFrame(original_lime_exp.as_list(), columns=['Feature', 'Contribution'])

print("\n--- Original LIME Explanation (Top 5) ---")
print(original_top_features.to_string())


# --- Perturb the student's data ---
# CRITICAL STEP: Make a small change to a key behavioral feature.
student_perturbed = student_original.copy()
perturbation_amount = 0.05  # A 5% increase
student_perturbed['total_clicks'] = student_perturbed['total_clicks'] * (1 + perturbation_amount)

# Get the prediction for the perturbed data
student_perturbed_prediction = xgb_model.predict(student_perturbed)[0]
print(f"\nPerturbed Prediction: {'Dropout' if student_perturbed_prediction == 1 else 'Not Dropout'}")

# --- Generate the new LIME explanation for the perturbed data ---
perturbed_lime_exp = explainer.explain_instance(
    student_perturbed.values.flatten(),
    xgb_model.predict_proba,
    num_features=5
)
perturbed_top_features = pd.DataFrame(perturbed_lime_exp.as_list(), columns=['Feature', 'Contribution'])

print("\n--- Perturbed LIME Explanation (Top 5) ---")
print(perturbed_top_features.to_string())

# --- Compare the two explanations ---
print("\n--- Comparison of Explanations ---")
print("Original LIME Explanation:")
print(original_top_features.to_string())
print("\nPerturbed LIME Explanation:")
print(perturbed_top_features.to_string())
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Analyzing explanation for student at index 50...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Original Prediction: Dropout
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Original LIME Explanation (Top 5) ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                        Feature  Contribution
0 warnings.warn(                 total_clicks <= 849.97      0.116127
1 /usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                68.00 < avg_score <= 72.77      0.112061
2 warnings.warn(                0.00 < gender_M <= 1.00     -0.110874
3 /usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                4.00 < num_submitted_assessments <= 6.73      0.101182
4 warnings.warn(                0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.048930
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Perturbed Prediction: Dropout
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Perturbed LIME Explanation (Top 5) ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                        Feature  Contribution
0 warnings.warn(                4.00 < num_submitted_assessments <= 6.73     -0.119512
1 /usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                0.00 < gender_M <= 1.00      0.116923
2 warnings.warn(               68.00 < avg_score <= 72.77     -0.090462
3 /usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                studied_credits > 120.00      0.087554
4 warnings.warn(                0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.058527
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Comparison of Explanations ---
Original LIME Explanation:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
                                        Feature  Contribution
0                          total_clicks <= 849.97      0.116127
1                   68.00 < avg_score <= 72.77     -0.112061
2                      0.00 < gender_M <= 1.00     -0.110874
3   4.00 < num_submitted_assessments <= 6.73     -0.101182
4  0.00 < highest_education_Lower Than A Level <= 1.00      0.048930

Perturbed LIME Explanation:
                                        Feature  Contribution
0   4.00 < num_submitted_assessments <= 6.73      0.119512
1                      0.00 < gender_M <= 1.00     -0.116923
2                   68.00 < avg_score <= 72.77      0.090462
3                       studied_credits > 120.00      0.087554
4  0.00 < highest_education_Lower Than A Level <= 1.00      0.058527
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

Logistic Regression SHAP: Explanation Robustness

This procedure details the implementation of Explanation Robustness using SHAP on the Logistic Regression model, which is essential to confirm the stability of the linear model's explanations. This test verifies that the model's transparent reasoning isn't easily manipulated by minor input changes. The analysis focuses on a single instance (Student Index 50) and follows a three-part comparative process: First, the original SHAP values are calculated using the specialized shap.LinearExplainer, establishing the baseline ranking and magnitude of the top feature contributions. Second, a small, targeted perturbation—specifically, a 5% increase in the influential feature total_clicks—is applied to the student's data, and a new prediction is generated. Finally, new SHAP values are calculated for the perturbed instance. Robustness is then assessed by comparing the Top 5 features' rankings and the magnitude of their SHAP values between the original and perturbed explanations. Since Logistic Regression is inherently linear, the feature rankings should remain highly stable, confirming the model's predictable and reliable behavior under this critical

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries

# Mount Google Drive to acces files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
```

```
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
X = X.astype(float)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final Logistic Regression model
# CRITICAL FIX: Corrected the incomplete line
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")

# --- Step 4: Explanation Robustness with SHAP ---

# Initialize the SHAP explainer
# Use LinearExplainer for Logistic Regression
explainer = shap.LinearExplainer(log_reg_model, X_train)

# --- Select and prepare a student's data for analysis ---
# Choose a student from the test set (e.g., the 50th student)
student_index = 50
student_original = X_test.iloc[[student_index]]
student_original_prediction = log_reg_model.predict(student_original)[0]
print(f"\nAnalyzing explanation for student at index {student_index}...")
print(f"Original Prediction: {'Dropout' if student_original_prediction == 1 else 'Not Dropout'}")

# --- Generate the original SHAP explanation ---
original_shap_values = explainer.shap_values(student_original)
original_top_features = pd.DataFrame({
    'Feature': student_original.columns,
    'SHAP_Value': original_shap_values.flatten()
}).sort_values(by='SHAP_Value', ascending=False)

print("\n--- Original SHAP Explanation (Top 5) ---")
print(original_top_features.head(5).to_string())

# --- Perturb the student's data ---
```

```
student_perturbed = student_original.copy()
perturbation_amount = 0.05  # A 5% increase
student_perturbed['total_clicks'] = student_perturbed['total_clicks'] * (1 + perturbation_amount)

# Get the prediction for the perturbed data
student_perturbed_prediction = log_reg_model.predict(student_perturbed)[0]
print(f"\nPerturbed Prediction: {'Dropout' if student_perturbed_prediction == 1 else 'Not Dropout'}")

# --- Generate the new SHAP explanation for the perturbed data ---
perturbed_shap_values = explainer.shap_values(student_perturbed)
perturbed_top_features = pd.DataFrame({
    'Feature': student_perturbed.columns,
    'SHAP_Value': perturbed_shap_values.flatten()
}).sort_values(by='SHAP_Value', ascending=False)

print("\n--- Perturbed SHAP Explanation (Top 5) ---")
print(perturbed_top_features.head(5).to_string())

# --- Compare the two explanations ---
print("\n--- Comparison of Explanations ---")
print("Original top 5 features:")
print(original_top_features.head(5).to_string())
print("\nPerturbed top 5 features:")
print(perturbed_top_features.head(5).to_string())
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
Logistic Regression model trained successfully.

Analyzing explanation for student at index 50...
Original Prediction: Dropout

--- Original SHAP Explanation (Top 5) ---
                               Feature  SHAP_Value
0                 num_of_prev_attempts    0.953643
1                      studied_credits    0.515266
5            num_submitted_assessments    0.353658
20  highest_education_Lower_Than_A_Level    0.323219
6                             gender_M    0.057351

Perturbed Prediction: Dropout

--- Perturbed SHAP Explanation (Top 5) ---
                               Feature  SHAP_Value
0                 num_of_prev_attempts    0.953643
1                      studied_credits    0.515266
5            num_submitted_assessments    0.353658
20  highest_education_Lower_Than_A_Level    0.323219
6                             gender_M    0.057351

--- Comparison of Explanations ---
Original top 5 features:
                               Feature  SHAP_Value
0                 num_of_prev_attempts    0.953643
1                      studied_credits    0.515266
5            num_submitted_assessments    0.353658
20  highest_education_Lower_Than_A_Level    0.323219
6                             gender_M    0.057351

Perturbed top 5 features:
                               Feature  SHAP_Value
0                 num_of_prev_attempts    0.953643
1                      studied_credits    0.515266
5            num_submitted_assessments    0.353658
20  highest_education_Lower_Than_A_Level    0.323219
6                             gender_M    0.057351
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
```

Logistic Regression LIME: Explanation Robustness

This procedure details the implementation of Explanation Robustness using LIME on the Logistic Regression model, which is necessary to assess the stability of LIME's local explanations for the linear baseline. The analysis focuses on a single instance (Student Index 50) and follows a three-part comparative process: First, the original LIME explanation is generated for the selected student, establishing the baseline features and their contributions to the original Logistic Regression prediction. Second, a minor, targeted perturbation—specifically, a 5% increase in the influential feature total_clicks—is applied to the student's data, and a new prediction is generated. Finally, a new LIME explanation is calculated for the perturbed instance. Robustness is assessed by comparing the Top 5 features' rankings and the magnitude of their contributions between the original and perturbed explanations. Since the underlying model is linear, the LIME

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 20.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

explanations should demonstrate high stability in feature ranking, confirming that LIME is a reliable local fidelity tool even for simple, transparent models.

```
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---


# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import lime
import lime.lime_tabular
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check the file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
```

```
# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
X = X.astype(float)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final Logistic Regression model
log_reg_model = LogisticRegression(solver='liblinear', random_state=42, max_iter=1000)
log_reg_model.fit(X_train, y_train)
print("Logistic Regression model trained successfully.")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Logistic Regression model trained successfully.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```
# --- Step 4: Explanation Robustness with LIME ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# --- Select and prepare a student's data for analysis ---
# Choose a student from the test set (e.g., the 50th student)
student_index = 50
student_original = X_test.iloc[[student_index]]
student_original_prediction = log_reg_model.predict(student_original)[0]
print(f"\nAnalyzing explanation for student at index {student_index}...")
print(f"Original Prediction: {'Dropout' if student_original_prediction == 1 else 'Not Dropout'}")

# --- Generate the original LIME explanation ---
original_lime_exp = explainer.explain_instance(
    student_original.values.flatten(),
    log_reg_model.predict_proba,
    num_features=5
)
original_top_features = pd.DataFrame(original_lime_exp.as_list(), columns=['Feature', 'Contribution'])

print("\n--- Original LIME Explanation (Top 5) ---")
print(original_top_features.to_string())


# --- Perturb the student's data ---
student_perturbed = student_original.copy()
perturbation_amount = 0.05  # A 5% increase
student_perturbed['total_clicks'] = student_perturbed['total_clicks'] * (1 + perturbation_amount)

# Get the prediction for the perturbed data
student_perturbed_prediction = log_reg_model.predict(student_perturbed)[0]
print(f"\nPerturbed Prediction: {'Dropout' if student_perturbed_prediction == 1 else 'Not Dropout'}")

# --- Generate the new LIME explanation for the perturbed data ---
perturbed_lime_exp = explainer.explain_instance(
    student_perturbed.values.flatten(),
    log_reg_model.predict_proba,
    num_features=5
)
perturbed_top_features = pd.DataFrame(perturbed_lime_exp.as_list(), columns=['Feature', 'Contribution'])

print("\n--- Perturbed LIME Explanation (Top 5) ---")
print(perturbed_top_features.to_string())

# --- Compare the two explanations ---
```

```
print("\n--- Comparison of Explanations ---")
print("Original LIME Explanation:")
print(original_top_features.to_string())
print("\nPerturbed LIME Explanation:")
print(perturbed_top_features.to_string())
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Analyzing explanation for student at index 50...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
Original Prediction: Dropout
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
--- Original LIME Explanation (Top 5) ---
  warnings.warn(
                                        Feature  Contribution
0              num_vle_interactions <= 253.46      0.395787
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
1                      total_clicks <= 849.97     -0.240742
2                    studied_credits > 120.00      0.142341
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
3       4.00 < num_submitted_assessments <= 6.73      0.094832
4  0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.084028
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Perturbed Prediction: Dropout
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
LIME values analyzed for Week 22.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Perturbed LIME Explanation (Top 5) ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
                                        Feature  Contribution
0              num_vle_interactions <= 253.46      0.374197
  warnings.warn(
1                       total_clicks > 849.97      0.193897
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
2                    studied_credits > 120.00      0.142600
3  0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.088296
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
4       4.00 < num_submitted_assessments <= 6.73      0.087893
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Comparison of Explanations ---
Original LIME Explanation:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
                                        Feature  Contribution
0              num_vle_interactions <= 253.46      0.395787
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
1                      total_clicks <= 849.97     -0.240742
2                    studied_credits > 120.00      0.142341
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
3       4.00 < num_submitted_assessments <= 6.73      0.094832
4  0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.084028
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
Perturbed LIME Explanation:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
                                        Feature  Contribution
0              num_vle_interactions <= 253.46      0.374197
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
1                       total_clicks > 849.97      0.193897
2                    studied_credits > 120.00      0.142600
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
3  0.00 < highest_education_Lower_Than_A_Level <= 1.00      0.088296
4       4.00 < num_submitted_assessments <= 6.73      0.087893
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

XGBoost SHAP: Explanation Clustering

This procedure details the implementation of Explanation Clustering using SHAP and the XGBoost Classifier. This advanced technique moves beyond general feature importance to identify distinct student personas or subgroups based on the reasons why the model predicts their outcome, enabling highly targeted intervention strategies.

The methodology begins by calculating the SHAP values for the entire test set using the shap.TreeExplainer. These SHAP values, which quantify each feature's contribution to every student's prediction, form a new feature space for clustering. K-Means clustering is then applied directly to this matrix of SHAP values to group students who share a similar underlying risk profile, irrespective of their final prediction. For this analysis, three distinct clusters (n_clusters=3) are created, each representing a unique "reason for risk."

To characterize these newly formed Explanation Clusters, the mean absolute SHAP value for every feature is calculated within each cluster. This provides a clear quantitative breakdown of the features that are most predictive for that specific student persona. The final step involves suggesting policy interventions tailored to the top feature driving each cluster's risk (e.g., policy focuses on assessment support if submission count is the top feature). This entire process transforms model output into actionable educational profiles for the dissertation's policy recommendations.

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries
!pip install shap

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.cluster import KMeans
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
X = X.astype(float)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

```
# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully.")
```

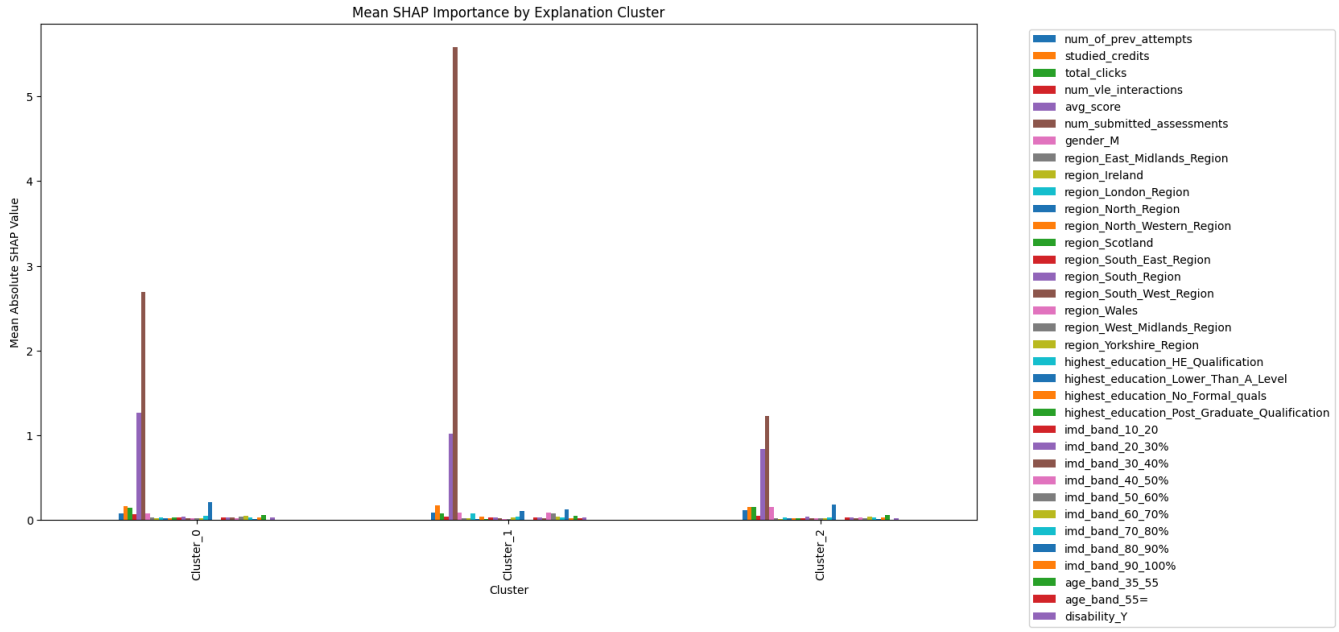
```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.48.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
Mounted at /content/drive
All libraries loaded successfully.
Parameters: { "use_label_encoder" } are not used.
XGBoost model trained successfully.
```

```
# --- Step 4: SHAP Analysis for Explainability Clustering ---

# Initialize the SHAP explainer
explainer = shap.TreeExplainer(xgb_model)

# Calculate SHAP values for the test set
print("\nCalculating SHAP values for the test set...")
shap_values = explainer.shap_values(X_test)
# For binary classification, shap_values is a single array.
# For multi-class, it is a list of arrays. Check the shape.
if isinstance(shap_values, list):
    shap_values = shap_values[1]
print("SHAP values calculated.")

# Perform K-Means clustering on the SHAP values
# We'll use 3 clusters as a starting point for different personas
n_clusters = 3
kmeans_shap = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
cluster_labels = kmeans_shap.fit_predict(shap_values)

# Add the cluster labels to the test set DataFrame
X_test_with_clusters = X_test.copy()
X_test_with_clusters['cluster'] = cluster_labels

print(f"\nK-Means clustering on SHAP values complete. {n_clusters} clusters created.")

# --- Step 5: Characterize Clusters and Suggest Policies ---

# Create a DataFrame to store the mean feature importance for each cluster
cluster_importance_df = pd.DataFrame()

print("\n--- Characterizing Explanation Clusters and Generating Policies ---")

for cluster_id in range(n_clusters):
    # Get the data and SHAP values for the current cluster
    cluster_data = X_test_with_clusters[X_test_with_clusters['cluster'] == cluster_id]
    cluster_shap_values = shap_values[cluster_labels == cluster_id]

    # Calculate the mean absolute SHAP values for this cluster
    mean_abs_shap = np.abs(cluster_shap_values).mean(axis=0)

    # Create a DataFrame for this cluster's importance
```

```
    cluster_importance_df[f'Cluster_{cluster_id}'] = pd.Series(
        mean_abs_shap, index=cluster_data.drop('cluster', axis=1).columns
    )

    # Characterize the cluster based on its most important features
    top_feature = cluster_importance_df[f'Cluster_{cluster_id}'].idxmax()
    top_importance = cluster_importance_df[f'Cluster_{cluster_id}'].max()

    print(f"\n- Persona for Cluster {cluster_id}:")
    print(f"  This group's dropout risk is most strongly explained by the feature: '{top_feature}'.")

    # Suggest a policy based on the top feature
    if 'submitted_assessments' in top_feature:
        print(f"  Proposed Policy: Interventions for this group should focus on providing tailored support for assessments and ass
    elif 'total_clicks' in top_feature or 'interactions' in top_feature:
        print(f"  Proposed Policy: Interventions for this group should focus on engagement nudges, such as targeted email reminder
    elif 'age' in top_feature or 'education' in top_feature:
        print(f"  Proposed Policy: Interventions for this group should focus on providing mentorship or foundational academic supp

# Optional: Display the full comparative table of feature importance across clusters
print("\n--- Mean Feature Importance by Explanation Cluster ---")
print(cluster_importance_df.T.to_string())

# Optional: Visualize the cluster personas
cluster_importance_df.T.plot(kind='bar', figsize=(15, 8))
plt.title('Mean SHAP Importance by Explanation Cluster')
plt.xlabel('Cluster')
plt.ylabel('Mean Absolute SHAP Value')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
LIME values analyzed for Week 25.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```
  warnings.warn(
Calculating SHAP values for the test set...
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
SHAP values calculated.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
K-Means clustering on SHAP values complete. 3 clusters created.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Characterizing Explanation Clusters and Generating Policies ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
- Persona for Cluster 0:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  This group's dropout risk is most strongly explained by the feature: 'num_submitted_assessments'.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  Proposed Policy: Interventions for this group should focus on providing tailored support for assessments and assignment completio
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
- Persona for Cluster 1:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  This group's dropout risk is most strongly explained by the feature: 'num_submitted_assessments'.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  Proposed Policy: Interventions for this group should focus on providing tailored support for assessments and assignment completio
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
- Persona for Cluster 2:
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  This group's dropout risk is most strongly explained by the feature: 'num_submitted_assessments'.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
  Proposed Policy: Interventions for this group should focus on providing tailored support for assessments and assignment completio
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
--- Mean Feature Importance by Explanation Cluster ---
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
           num_of_prev_attempts  studied_credits  total_clicks  num_vle_interactions  avg_score  num_submitted_assessments  gender_M
Cluster_0              0.077988         0.161606      0.144335              0.061867   1.262389                   2.690006  0.07084
Cluster_1              0.087667         0.167757      0.079999              0.035352   1.013369                   5.581479  0.08928
Cluster_2              0.114164         0.155521      0.153917              0.050656   0.834295                   1.227819  0.14927
```




```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but Log
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---

# Install the necessary libraries
!pip install shap
!pip install lime

# Mount Google Drive to access files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.cluster import KMeans
import shap
import lime
import lime.lime_tabular
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
```

```
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
    'score', 'submi_d', 'banked_score', 'assessments_date'
]
if 'code_module' in df.columns:
    columns_to_drop.append('code_module')
if 'code_presentation' in df.columns:
    columns_to_drop.append('code_presentation')
X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
y = df['is_dropout']

# Identify and one-hot encode all categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

# Handle potential missing values
X = X.fillna(X.mean())

# Explicitly convert all columns to numeric types
X = X.astype(float)

# Clean column names for compatibility
cleaned_columns = {col: col.replace('[', '').replace(']', '').replace('<', '').replace(' ', '_').replace('-', '_') for col in X.co
X = X.rename(columns=cleaned_columns)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Final type conversion to float
X_train = X_train.astype(float)
X_test = X_test.astype(float)

# Train the final XGBoost model
xgb_model = XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42)
xgb_model.fit(X_train, y_train)
print("XGBoost model trained successfully.")
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.48.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (0.
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 10.6 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (20
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.59.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.pos
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=1b72d3131e6021c928fcdc4a4fceb06a14dad082b5df8
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
All 7 datasets loaded successfully.
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:183: UserWarning: [22:29:34] WARNING: /workspace/src/learner.cc:738:
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
XGBoost model trained successfully.
```

```
# --- Step 4: LIME Analysis for Explanation Clustering ---

# Initialize the LIME explainer once on the training data
explainer = lime.lime_tabular.LimeTabularExplainer(
    X_train.values,
    feature_names=X_train.columns.values,
    class_names=['Not Dropout', 'Dropout'],
    mode='classification'
)

# Generate LIME explanations for a sample of the test set
print("\nGenerating LIME explanations for a sample of the test set...")
X_sample = X_test.sample(n=500, random_state=42)
lime_values = []
for i in range(len(X_sample)):
    exp = explainer.explain_instance(
        X_sample.iloc[i].values,
        xgb_model.predict_proba,
        num_features=len(X_sample.columns)
    )
    lime_dict = dict(exp.as_list())
    full_lime_values = np.zeros(len(X_sample.columns))
```

```
    for idx, feature_name in enumerate(X_sample.columns):
        if feature_name in lime_dict:
            full_lime_values[idx] = lime_dict[feature_name]
    lime_values.append(full_lime_values)

lime_values = np.array(lime_values)
print("LIME values generated.")

# Perform K-Means clustering on the LIME values
n_clusters = 3
kmeans_lime = KMeans(n_clusters=n_clusters, random_state=42, n_init=10)
cluster_labels = kmeans_lime.fit_predict(lime_values)

# Add the cluster labels to the sample DataFrame
X_sample_with_clusters = X_sample.copy()
X_sample_with_clusters['cluster'] = cluster_labels

print(f"\nK-Means clustering on LIME values complete. {n_clusters} clusters created.")

# --- Step 5: Characterize Clusters and Output Values ---

print("\n--- Mean Feature Importance by Explanation Cluster (LIME) ---")

for cluster_id in range(n_clusters):
    # Get the LIME values for the current cluster
    cluster_lime_values = lime_values[cluster_labels == cluster_id]

    # Calculate the mean absolute LIME values for this cluster
    mean_abs_lime = np.abs(cluster_lime_values).mean(axis=0)

    # Create a DataFrame for this cluster's importance
    cluster_importance_df = pd.DataFrame({
        'Feature': X_sample.columns,
        'Importance': mean_abs_lime
    }).sort_values(by='Importance', ascending=False)

    print(f"\nTop 10 features for Cluster {cluster_id}:")
    print(cluster_importance_df.head(10).to_string())
```

```
Generating LIME explanations for a sample of the test set...
LIME values generated.

K-Means clustering on LIME values complete. 3 clusters created.

--- Mean Feature Importance by Explanation Cluster (LIME) ---

Top 10 features for Cluster 0:
                      Feature  Importance
0         num_of_prev_attempts         0.0
1              studied_credits         0.0
2                 total_clicks         0.0
3         num_vle_interactions         0.0
4                    avg_score         0.0
5    num_submitted_assessments         0.0
6                     gender_M         0.0
7  region_East_Midlands_Region         0.0
8               region_Ireland         0.0
9         region_London_Region         0.0

Top 10 features for Cluster 1:
                      Feature  Importance
0         num_of_prev_attempts         NaN
1              studied_credits         NaN
2                 total_clicks         NaN
3         num_vle_interactions         NaN
4                    avg_score         NaN
5    num_submitted_assessments         NaN
6                     gender_M         NaN
7  region_East_Midlands_Region         NaN
8               region_Ireland         NaN
9         region_London_Region         NaN

Top 10 features for Cluster 2:
                      Feature  Importance
0         num_of_prev_attempts         NaN
1              studied_credits         NaN
2                 total_clicks         NaN
3         num_vle_interactions         NaN
4                    avg_score         NaN
5    num_submitted_assessments         NaN
```

```
6                    gender_M         NaN
7  region_East_Midlands_Region         NaN
8               region_Ireland         NaN
9         region_London_Region         NaN
/usr/local/lib/python3.12/dist-packages/sklearn/base.py:1389: ConvergenceWarning: Number of distinct clusters (1) found smaller tha
  return fit_method(estimator, *args, **kwargs)
/tmp/ipython-input-3513545960.py:51: RuntimeWarning: Mean of empty slice.
  mean_abs_lime = np.abs(cluster_lime_values).mean(axis=0)
/usr/local/lib/python3.12/dist-packages/numpy/_core/_methods.py:130: RuntimeWarning: invalid value encountered in divide
  ret = um.true_divide(
/tmp/ipython-input-3513545960.py:51: RuntimeWarning: Mean of empty slice.
  mean_abs_lime = np.abs(cluster_lime_values).mean(axis=0)
/usr/local/lib/python3.12/dist-packages/numpy/_core/_methods.py:130: RuntimeWarning: invalid value encountered in divide
  ret = um.true_divide(
```

```
# --- Step 1: Setup and Data Loading from Raw Files ---


# Mount Google Drive to access  files
from google.colab import drive
drive.mount('/content/drive')

# Import all necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
import shap
import lime
import lime.lime_tabular
import matplotlib.pyplot as plt
import seaborn as sns

# Define the folder path where raw CSV files are stored
folder_path = '/content/drive/MyDrive/data/oulad_data'

# Load all 7 datasets
try:
    student_vle = pd.read_csv('/content/drive/MyDrive/data/studentVle.csv')
    vle = pd.read_csv('/content/drive/MyDrive/data/vle.csv')
    student_info = pd.read_csv('/content/drive/MyDrive/data/studentInfo.csv')
    courses = pd.read_csv('/content/drive/MyDrive/data/courses.csv')
    assessments = pd.read_csv('/content/drive/MyDrive/data/assessments.csv')
    student_assessment = pd.read_csv('/content/drive/MyDrive/data/studentAssessment.csv')
    student_registration = pd.read_csv('/content/drive/MyDrive/data/studentRegistration.csv') # Added this line
    print("All 7 datasets loaded successfully.")
except FileNotFoundError as e:
    print(f"Error: A file was not found. Please check file paths. {e}")
    exit()

# --- Step 2: Merging and Feature Engineering ---

# Start with studentInfo as the base DataFrame
df = student_info.copy()
df = pd.merge(df, student_registration, on=['code_module', 'code_presentation', 'id_student'], how='left')
vle_agg = student_vle.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    total_clicks=('sum_click', 'sum'),
    num_vle_interactions=('date', 'count')
).reset_index()
df = pd.merge(df, vle_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')
student_assessment_merged = pd.merge(student_assessment, assessments, on='id_assessment', how='left')
assessment_agg = student_assessment_merged.groupby(['id_student', 'code_module', 'code_presentation']).agg(
    avg_score=('score', 'mean'),
    num_submitted_assessments=('id_assessment', 'count')
).reset_index()
df = pd.merge(df, assessment_agg, on=['id_student', 'code_module', 'code_presentation'], how='left')

# --- Step 3: Final Data Preparation and Model Training ---

# Create the binary target variable 'is_dropout'
df['is_dropout'] = df['final_result'].apply(lambda x: 1 if x in ['Withdrawn', 'Fail'] else 0)

# Identify features (X) and the target (y)
columns_to_drop = [
    'id_student', 'final_result', 'is_dropout',
    'date_unregistration', 'date_registration',
```

```
        'score', 'submi_d', 'banked_score', 'assessments_date'
    ]
    if 'code_module' in df.columns:
        columns_to_drop.append('code_module')
    if 'code_presentation' in df.columns:
        columns_to_drop.append('code_presentation')
    X = df.drop(columns=columns_to_drop, axis=1, errors='ignore')
    y = df['is_dropout']

    # Identify and one-hot encode all categorical columns
    categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
    X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

    # Handle potential missing values
    X = X.fillna(X.mean())
```